

Tấn công giấu tin FHSS trong âm thanh

FHSS Audio Steganography Attack Lab

1 Overview

Mục tiêu của bài lab là khảo sát tính mong manh của kỹ thuật giấu tin trong âm thanh sử dụng FHSS. Trong bài này, thông điệp đã được nhúng vào file stego.wav bằng các thay đổi rất nhỏ trên mẫu âm thanh. Sinh viên sẽ thử các phép tấn công như lượng tử hóa, thêm nhiễu, hoặc xóa các bit thấp để làm sai thông điệp khi khôi phục, đồng thời không được phá hỏng định dạng WAV.

Sau khi hoàn thành bài lab, sinh viên cần hiểu:

- Vì sao một thay đổi nhỏ trong mẫu âm thanh có thể làm sai bit giấu tin.
- Sự khác biệt giữa phá file âm thanh và tấn công có kiểm soát.
- Cách đánh giá một tấn công: có làm hỏng thông điệp không và file audio còn hợp lệ không.
- Cách thử nhiều tham số tấn công thay vì chỉ chạy một lệnh duy nhất.

2 Lab Environment

Bài lab chạy trong Labtainer framework. Từ thư mục labtainer-student, khởi động bài lab bằng lệnh:

```
labtainer fhss_attack
```

Sau khi lab khởi động, một terminal container tên fhss sẽ được mở. Các thao tác trong phần Lab Tasks được thực hiện trong terminal này.

3 Setup

Trong terminal fhss, kiểm tra các file được cung cấp:

```
ls -l
```

Các file chính gồm:

File	Vai trò
cover.wav	File âm thanh gốc, dùng làm tham chiếu khi khôi phục bit.
stego.wav	File âm thanh đã được giấu thông điệp bằng FHSS.
recover.py	Công cụ khôi phục thông điệp từ một file WAV khi biết key và frame size.
baseline_check.py	Kiểm tra rằng stego.wav ban đầu vẫn khôi phục được thông điệp gốc.
attack.py	Công cụ tạo attacked.wav bằng các phương pháp tấn công khác nhau.
damage_check.py	Kiểm tra attacked.wav có làm hỏng thông điệp khi khôi phục hay không.
quality_check.py	Kiểm tra attacked.wav vẫn là file WAV hợp lệ và không bị phá hỏng quá mức.

4 Lab Tasks

4.1 Task 1: Kiểm tra trạng thái ban đầu của file stego

Trước khi tấn công, cần xác nhận rằng file stego.wav thật sự chứa thông điệp có thể khôi phục được. Chạy:

```
./baseline_check.py
```

Kết quả mong đợi có dạng:

```
RECOVER_RUN_OK
INPUT_FILE=stego.wav
OUTPUT_FILE=original.txt
RECOVERED_TEXT=FHSSHARD1
BASELINE_OK
```

Nếu thấy BASELINE_OK, điều đó nghĩa là thông điệp gốc vẫn còn khôi phục được từ stego.wav. Đây là mốc so sánh trước khi tấn công.

4.2 Task 2: Tạo một file âm thanh bị tấn công

Xem các tùy chọn của công cụ tấn công:

```
./attack.py -h
```

Công cụ hỗ trợ một số phương pháp tấn công:

- quantize: lượng tử hóa mẫu âm thanh theo một bước nhất định.
- noise: thêm nhiễu ngẫu nhiên nhỏ vào mẫu âm thanh.
- lsb-clear: xóa một số bit thấp của mẫu âm thanh.

Hãy thử nhiều tham số khác nhau. Ví dụ, với phương pháp quantize:

```
./attack.py --input stego.wav --output attacked.wav --method quantize --step 2
```

```
./attack.py --input stego.wav --output attacked.wav --method quantize --step 4
```

```
./attack.py --input stego.wav --output attacked.wav --method quantize --step 8
```

Nếu file attacked.wav được tạo thành công, output sẽ có dòng:

Labtainers

ATTACK_CREATED

Bạn cũng có thể thử các phương pháp khác:

```
./attack.py --input stego.wav --output attacked.wav --method noise --amplitude 4
```

```
./attack.py --input stego.wav --output attacked.wav --method lsb-clear --bits 3
```

4.3 Task 3: Kiểm tra tấn công có làm hỏng thông điệp không

Sau mỗi lần tạo attacked.wav, kiểm tra xem thông điệp còn khôi phục đúng hay không:

```
./damage_check.py
```

Nếu tấn công có tác dụng, kết quả sẽ có dòng:

```
DAMAGE_OK
```

Nếu vẫn thấy DAMAGE_FAIL, nghĩa là thông điệp còn khôi phục được. Khi đó hãy thử tăng mức tấn công, ví dụ tăng step của quantize, tăng amplitude của noise, hoặc tăng số bit trong lsb-clear.

4.4 Task 4: Kiểm tra chất lượng file bị tấn công

Một tấn công hợp lệ không được chỉ đơn giản là xóa file, làm rỗng file, hoặc phá định dạng WAV. Sau khi đã có DAMAGE_OK, chạy:

```
./quality_check.py
```

Nếu attacked.wav vẫn hợp lệ và mức thay đổi không quá lớn, kết quả sẽ có dòng:

```
QUALITY_OK
```

Nếu QUALITY_FAIL, tấn công có thể quá yếu hoặc quá mạnh. Hãy điều chỉnh tham số và thử lại.

4.5 Quan sát kết quả

Sau khi chạy các bước trên, có thể xem các file được sinh ra:

```
cat original.txt
cat attacked_result.txt
head attacked.log
```

original.txt nên chứa thông điệp gốc. attacked_result.txt thường sẽ không còn là thông điệp đúng nếu tấn công thành công.

5 Checkwork

Khi làm bài, có thể chạy checkwork từ terminal host để xem tiến độ. Các kết quả mong đợi là:

Checkwork	Lệnh liên quan	Ý nghĩa
g_baseline	./baseline_check.py	File stego ban đầu khôi phục được thông điệp gốc.
g_attack	./attack.py ...	Sinh viên đã tạo file attacked.wav bằng công cụ tấn công.
g_damage	./damage_check.py	File attacked.wav làm thông điệp bị hỏng khi khôi phục.
g_quality	./quality_check.py	File attacked.wav vẫn là WAV hợp lệ và không bị phá quá mức.

Khi hoàn thành toàn bộ bài, checkwork nên hiện:

```
Y - g_baseline
Y - g_attack
Y - g_damage
Y - g_quality
```

6 Hints and Troubleshooting

- Nếu DAMAGE_FAIL, tấn công chưa đủ mạnh. Hãy thử quantize với step lớn hơn, ví dụ 8 hoặc 16.
- Nếu QUALITY_FAIL, tấn công có thể quá mạnh hoặc làm thay đổi file quá nhiều. Hãy giảm step, amplitude hoặc số bit bị xóa.
- Nếu thiếu attacked.wav, hãy chạy lại attack.py trước khi chạy damage_check.py hoặc quality_check.py.
- Không chỉnh sửa trực tiếp stego.wav. File output của tấn công nên là attacked.wav.
- Không xóa hoặc làm rỗng file attacked.wav để qua bài. quality_check.py sẽ phát hiện file không hợp lệ.

7 Submission

Sau khi hoàn thành bài lab, quay lại terminal host dùng để khởi động lab và chạy:

```
stoplab fhss_attack
```

Labtainer sẽ hiển thị đường dẫn tới file kết quả đã được nén. Nộp file đó theo yêu cầu của giảng viên.

8 Command Summary

Một chuỗi lệnh mẫu để kiểm tra bài lab là:

```
./baseline_check.py
./attack.py --input stego.wav --output attacked.wav --method quantize --step 8
./damage_check.py
./quality_check.py
```

Trong quá trình làm bài, sinh viên nên thử nhiều tham số khác nhau thay vì chỉ dùng ngay một giá trị cố định.